

Tema: Estimaciones en Ambientes Ágiles

Temas Abordados (Índice)

Tema 1: Fundamentos y Filosofía de la Estimación Ágil

¿Para qué estimamos? La diferencia conceptual entre Estimación, Plan y Compromiso.
El Cono de Incertidumbre (Cone of Uncertainty) y la variabilidad de requerimientos.
El valor intrínseco del "proceso de hacer estimaciones".

Tema 2: Estimación Relativa y Story Points

Limitaciones cognitivas de la estimación absoluta y el poder de la analogía.
¿Qué es un Story Point (SP)? Su definición como amalgama de esfuerzo, complejidad y riesgo/duda.
Escala de medición no lineales (Fibonacci, potencias de 2, T-Shirt sizing) y el crecimiento exponencial de la complejidad.
La Triangulación como mecanismo formal de calibración y consistencia.

Tema 3: Dinámica Colaborativa: Planning Poker

Orígenes y fundamentos en la "Sabiduría Colectiva" (Wisdom of Crowds).
Algoritmo detallado paso a paso del Planning Poker.
Decodificación y semántica de las cartas (los valores extremos y cartas especiales).
El costo de coordinación y transacción de la planificación grupal.

Tema 4: Progreso y Capacidad de Ejecución: Velocidad y Capacidad

Definición formal de Velocidad (Velocity) y la regla del "todo o nada".
El mecanismo de autocorrección de la Velocidad frente a desvíos de estimación.
Capacidad (Capacity) e iteraciones.
El Factor de Dedicación (Focus Factor) y su cálculo en la planificación de Sprints.

Tema 5: El Enfoque Kanban y la Previsión Probabilística

La estimación detallada como desperdicio (waste) y el movimiento #NoEstimates.
Métricas de flujo en Kanban: Lead Time, Touch Time, Throughput y Eficiencia del Ciclo.
Previsión probabilística mediante simulaciones de Monte Carlo.

Tema 1: Fundamentos y Filosofía de la Estimación Ágil

¿Para qué estimamos? Estimaciones vs. Planes vs. Compromisos

En la gestión tradicional y en los enfoques de ingeniería convencionales, suele existir una peligrosa **confusión entre estimaciones, planes y compromisos**. Un ingeniero en sistemas debe diferenciar estos conceptos con precisión:

Estimación: Es una valoración técnica del tamaño, esfuerzo, costo o cronograma de un software que se basa en información incompleta o vaga y que, por definición, no es precisa. Representa una aproximación probabilística de la realidad, no un número rígido.

Compromiso: Es una promesa de entrega de cierta funcionalidad en una fecha y con una calidad dadas. El material de la cátedra advierte con firmeza: "Si las estimaciones se utilizan como compromisos, son muy peligrosas y perjudiciales para cualquier organización". A mayor diferencia entre lo estimado y lo planeado, mayor es el riesgo sistémico del proyecto.

Plan: Es la determinación de cómo se van a utilizar los recursos y los plazos para alcanzar una meta, pero el plan no tiene que ser idéntico a lo estimado.

En los procesos de desarrollo ágiles, las estimaciones sirven principalmente como una **protección** para el equipo de desarrollo frente a compromisos poco realistas y como una herramienta de retroalimentación temprana para evaluar si el alcance del trabajo planificado es factible dentro de las restricciones físicas de capacidad y tiempo.

El Cono de Incertidumbre y la Variabilidad de Requerimientos

La ingeniería de software convencional asume de forma prescriptiva que es posible planificar un proyecto de manera determinista al inicio. Sin embargo, la realidad empírica demuestra que el software se comporta como un "blanco móvil" debido a que los requerimientos son inherentemente inestables y cambian dinámicamente conforme avanza el desarrollo.

Este fenómeno se conceptualiza formalmente a través del Cono de Incertidumbre (popularizado por Steve McConnell y Barry Boehm):

En las fases de arranque o iniciales del proyecto (donde solo se cuenta con definiciones de requerimientos a alto nivel), existe un nivel extremo de variabilidad y desconocimiento. Si la estimación inicial del esfuerzo requerido se denota como un valor esperado $\$x\$$, el rango real de variación del esfuerzo medido al final del proyecto puede oscilar entre $\$0.25x\$$ (sobrestimación extrema) y $\$4.0x\$$ (subestimación severa).

Aceptar plazos y presupuestos fijos e inmutables en la parte ancha de este cono genera presiones excesivas, forzando a los desarrolladores a tomar atajos técnicos que reducen de forma crítica la calidad interna del producto y omiten actividades indispensables de verificación.

Por ende, el enfoque ágil no busca la obsesión por la precisión exacta al inicio, sino la adaptación iterativa de las estimaciones a medida que el cono se estrecha gracias a que se genera mayor conocimiento real sobre el dominio de aplicación y la tecnología utilizada.

El Valor del "Proceso de Hacer Estimaciones"

Uno de los axiomas fundamentales de la agilidad establece que *"lo más beneficioso en las estimaciones es el proceso de hacerlas"*. La estimación en entornos ágiles no es una actividad en la que un experto se encierra de forma aislada a calcular números. Por el contrario, se concibe como una **actividad colaborativa y de debate técnico** donde el equipo completo de desarrollo (programadores, testers, analistas, arquitectos) analiza y discute detalladamente cada requerimiento con el Product Owner (PO).

Este proceso de discusión e intercambio es sumamente valioso porque:

- Permite la transferencia y socialización de conocimiento de dominio y arquitectónico entre los miembros experimentados y los menos familiarizados.
- Ayuda a identificar malentendidos críticos de alcance de forma sumamente temprana.
- Permite descubrir riesgos ocultos y supuestos tecnológicos erróneos antes de que se comience a escribir la primera línea de código.

Tema 2: Estimación Relativa y Story Points

Limitaciones de la Estimación Absoluta y el Poder de la Analogía

Los seres humanos somos notoriamente deficientes al intentar realizar estimaciones en términos absolutos (como determinar de forma directa cuántas horas de trabajo calendario tomará una tarea compleja). Sin embargo, la psicología cognitiva y la práctica de la ingeniería demuestran que las personas somos excelentes **comparando cosas** (estimación por analogía).

La estimación ágil aprovecha esta capacidad innata a través del principio de **Estimación Relativa**: en lugar de estimar de forma absoluta, comparamos la nueva funcionalidad con requerimientos o características que ya han sido desarrollados en el pasado. Mike Cohn utiliza la analogía del "vaso de refresco": al pedir un vaso grande en un restaurante desconocido, no se sabe con exactitud cuántas onzas o mililitros contiene (medida absoluta), pero se sabe con total certeza que es más grande que el mediano y más pequeño que el extra-grande, y que saciará la sed en base a la experiencia previa (medida relativa).

Este enfoque de comparación relativa aporta múltiples ventajas:

- Es significativamente más rápido de realizar que la estimación analítica detallada de tareas individuales.
- Promueve una mejor dinámica grupal y pensamiento de equipo.
- Reduce los costos de transacción de la planificación al tolerar márgenes de imprecisión en las etapas tempranas del proyecto.

¿Qué es un Story Point (SP)?

Un **Story Point (SP)** es una unidad de medida específica y exclusiva de cada equipo utilizada para expresar el tamaño relativo global de una Historia de Usuario o funcionalidad.

Un Story Point no es una medida de tiempo (aunque erróneamente algunos equipos tiendan a equipararse de forma directa a horas o días ideales). En su lugar, el valor de un Story Point es una amalgama (o síntesis intuitiva) de tres dimensiones clave:

Complejidad: El nivel de dificultad técnica o conceptual asociado al diseño e implementación del requerimiento.

Esfuerzo: La cantidad física de trabajo requerida para codificar, probar e integrar la funcionalidad.

Riesgo o Duda: El nivel de incertidumbre, falta de familiaridad con la tecnología, o dependencias externas que podrían complicar el desarrollo de la tarea.

Dado que cada equipo calibra su propio "metro patrón" de Story Points de forma interna (lo que para un equipo de especialistas un SP representa cierto nivel de esfuerzo, para un equipo de menor experiencia puede representar el doble), los Story Points no son comparables entre diferentes equipos. Joshua Kerievsky sugirió jocosamente el término NUTs (Nebulous Units of Time - Unidades Nebulosas de Tiempo) para enfatizar su carácter abstracto e incorpóreo.

Escalas de Medición No Lineales y Crecimiento Exponencial de la Complejidad

Para estimar el tamaño relativo de las historias de usuario, los equipos deben adoptar una escala de medición formal. Las escalas de estimación más utilizadas en la industria son:

- La Secuencia de Fibonacci: 0, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89
- La Escala Fibonacci Modificada (SWEBOK / Cohn): 0, 1/2, 1, 2, 3, 5, 8, 13, 20, 40, 100
- La Serie Exponencial (2^n): 1, 2, 4, 8, 16, 32, 64
- Talles de Remeras (T-Shirt Sizing): S, M, L, XL, XXL

La adopción de escalas no lineales no es arbitraria; responde a dos principios científicos fundamentales:

Ley de Weber-Fechner (Percepción): A medida que un estímulo o cantidad física se hace más grande, se requiere una diferencia proporcionalmente mayor para que el cerebro humano la detecte. En estimación, argumentar si una historia de usuario grande equivale a 79 u 80 horas implica un nivel de precisión falso que excede la capacidad del proceso. Limitar la escala a valores espaciados de forma no lineal (como elegir entre 40 u 80) simplifica las discusiones y ahorra tiempo valioso.

Crecimiento exponencial de la complejidad: La complejidad de un sistema de software no escala linealmente con su tamaño. A medida que se añaden componentes o lógica de negocio, los puntos de interacción y canales de comunicación potenciales aumentan de manera exponencial, incrementando inherentemente el riesgo y la probabilidad de aparición de defectos. La escala Fibonacci refleja adecuadamente este comportamiento exponencial.

Regla de Cátedra Inflexible: Una vez elegida la escala de estimación para el proyecto, ésta NO se cambia. Modificar la escala a mitad del desarrollo equivale a "cambiar el metro patrón" de medición de la organización, invalidando de manera inmediata toda la base histórica de métricas de velocidad recopiladas en las iteraciones previas.

La Triangulación como Mecanismo de Calibración

Para garantizar la consistencia a largo plazo y evitar la distorsión del significado del Story Point (fenómeno conocido como la degradación del metro patrón), los equipos ágiles utilizan la Triangulación.

La triangulación consiste en estimar una nueva historia comparándola simultáneamente con un conjunto de historias de referencia ya estimadas y ejecutadas. El procedimiento práctico se detalla a continuación:

Se colocan de forma visual en una pizarra o pared tarjetas de historias reales divididas en columnas según sus puntos asignados (1, 2, 3, 5, 8, 13).

Cuando se presenta una nueva historia para estimar (por ejemplo, con una estimación preliminar de 5 SP), el equipo realiza un filtro de calidad de forma cruzada.

Se pregunta al grupo: "¿Esta nueva historia es sensiblemente más compleja o requiere mayor esfuerzo que las historias que están en la columna de 3? ¿Y es razonablemente más pequeña o sencilla que las historias ubicadas en la columna de 8?".

Si la respuesta a ambas preguntas es afirmativa, la estimación de 5 SP está correctamente triangulada e institucionalizada.

Tema 3: Dinámica Colaborativa: Planning Poker

Orígenes y Fundamentos en la "Sabiduría Colectiva"

El **Planning Poker** (diseñado originalmente por James Grenning en 2002 y popularizado internacionalmente por Mike Cohn) es el método colaborativo por excelencia empleado por los equipos de alto rendimiento para realizar estimaciones ágiles de tamaño y esfuerzo.

Este método funciona y resulta sumamente confiable porque combina científicamente tres técnicas fundamentales de estimación de proyectos:

Opinión de Expertos: Los participantes que estiman las tareas son las personas técnicas (desarrolladores, testers, etc.) que realmente van a realizar el trabajo, cumpliendo con la máxima de Jørgensen: "Las personas más competentes en resolver una tarea deben ser quienes la estimen".

Analogía: Se asienta sobre la estimación de tamaño relativo comparando los requerimientos de forma rápida.

Desagregación: Permite evaluar y desmenuzar requerimientos complejos o epopeyas (Epics) en escenarios individuales más pequeños para simplificar su análisis.

Además, la mecánica del Planning Poker mitiga activamente el sesgo de anclaje (donde la primera estimación expresada en voz alta por un miembro influyente o senior del equipo condiciona y sesga la opinión del resto del grupo, reduciendo la diversidad de análisis) al obligar a los participantes a votar de manera simultánea. También neutraliza el fenómeno del "pensamiento de grupo" (groupthink) y la holgazanería social al exigir el involucramiento activo de cada estimador.

Algoritmo Detallado del Planning Poker

Para garantizar la efectividad de la sesión (la cual en equipos maduros suele completarse en un rango de 45 minutos manteniendo niveles óptimos de concentración) [133], el juego se rige bajo el siguiente algoritmo de ejecución paso a paso [7, 8, 54, 55, 250]:

Prerrequisitos de Entrada:

- Contar con un Product Backlog priorizado por importancia de negocio por el Product Owner.
- Definir de forma previa una Base Story (Historia Canónica / Historia Base) como patrón común de escala (por ejemplo, definir que cierta historia simple de referencia equivale a 2 SP).
- Cada estimador técnico (miembro de desarrollo) dispone de un mazo físico o digital de cartas con la escala de estimación acordada. El Product Owner (PO) y el Scrum Master (SM) participan en la sesión, pero el PO no tiene carta de estimación ni voto, ya que su rol es clarificar el negocio, no estimar el esfuerzo técnico.

Pasos del Algoritmo:

Lectura: El moderador (usualmente el PO) lee en voz alta la descripción de la historia de usuario a estimar y explica las condiciones de satisfacción o casos de prueba de aceptación de alto nivel asociados.

Clarificación: Los estimadores discuten la historia y hacen preguntas técnicas al PO sobre su alcance y lógica de negocio. Para evitar que esta etapa se convierta en un debate infinito de diseño arquitectónico (lo que degradará la eficiencia del proceso), cualquier miembro del equipo puede activar un temporizador de arena de 2 minutos (sand timer) para limitar la discusión y forzar la votación.

Votación Privada (Sin Anclaje): Cada estimador selecciona mentalmente una carta que represente el tamaño de la historia en comparación con la Historia Base. Coloca la carta boca abajo en la mesa para garantizar el secreto del voto.

Exposición Simultánea: Una vez que todos los participantes han seleccionado su carta, el moderador indica que se revelen todas las estimaciones de manera simultánea dando la vuelta a las cartas al mismo tiempo.

Evaluación de Consenso:

Caso A: Existe Consenso. Si todos los estimadores revelan exactamente el mismo valor de puntos, ese número se consagra formalmente como la estimación de la historia de usuario en el Product Backlog. Se pasa al paso 7.

Caso B: Discrepancia. Si existen diferencias en los valores expuestos, se activa una fase de debate enfocado.

Discusión de los extremos: El moderador pide específicamente a los estimadores que votaron el valor más alto y el valor más bajo que expliquen de forma racional sus supuestos y argumentos técnicos.

Decodificación y Semántica de las Cartas

El mazo de cartas de Planning Poker posee una semántica específica dictada por la cátedra para decodificar los supuestos del equipo de desarrollo durante la sesión:

Carta 0: Indica que la funcionalidad ya se encuentra implementada en el sistema de manera nativa, o bien que el esfuerzo, la complejidad y el riesgo asociados para su desarrollo son prácticamente nulos.

Cartas 1/2, 1, 2 y 3: Representan requerimientos pequeños, con baja complejidad y total familiaridad. Se consideran el "corazón de la iteración" por su fácil predictibilidad.

Carta 5: Representa una funcionalidad de tamaño medio. Es el valor esperado estándar de una historia de usuario equilibrada.

Carta 8: Es el límite de tamaño recomendado para una historia de usuario en un Sprint. Representa una funcionalidad grande que, si bien es realizable, exige que el equipo se

pregunte si es posible descomponerla verticalmente en rebanadas (vertical slices) más pequeñas y controlables.

Carta 13: Indica un requerimiento de tamaño muy elevado con alto nivel de incertidumbre. El equipo debe discutir obligatoriamente cómo dividir este elemento en historias más granulares.

Carta 20: Implica una complejidad o riesgo de negocio sumamente elevados. Exige que el Product Owner justifique de forma explícita el valor de negocio de semejante historia antes de que el equipo asuma cualquier esfuerzo conceptual.

Carta 40: No hay forma física de realizar un requerimiento de este tamaño dentro de un único Sprint de trabajo. Representa una Epopeya (Epic) que debe desglosarse en múltiples historias de usuario antes de poder ser considerada viable para una iteración.

Carta 100: Representa una confirmación técnica de que algo está sumamente mal concebido en los requerimientos. Indica una falta total de especificación o un nivel de riesgo técnico prohibitivo; se aconseja al equipo no iniciar el desarrollo de este requerimiento bajo ninguna circunstancia hasta que no sea completamente refinado y redefinido.

Carta ? (Interrogación): Representa una falta absoluta de conocimiento o comprensión sobre el requerimiento. Equivale a un "no tengo idea de qué se está pidiendo".

Carta de la Taza de Café: Indica que el nivel de cansancio cognitivo del equipo es tan elevado que las estimaciones están perdiendo rigor. Es una solicitud para realizar un descanso (receso) operativo antes de continuar estimando.

El Costo de Coordinación y Transacción en Equipos Grandes

Un análisis maduro de la estimación ágil exige comprender sus limitaciones en términos de teoría de la organización. En equipos de desarrollo pequeños (típicamente de menos de 10 ingenieros enfocados en un único producto), el Planning Poker es sumamente eficiente y dinámico.

Sin embargo, al intentar escalar la técnica a nivel corporativo (como en el caso analizado por David Anderson en Corbis, con 55 especialistas operando sobre 27 sistemas de TI concurrentes), la dinámica grupal del Planning Poker introduce una disfunción operativa grave debido a la fricción de coordinación:

Para lograr la "sabiduría colectiva", se requeriría la presencia física de los 55 profesionales en una misma sesión para evaluar historias que cruzan múltiples tecnologías y especialidades.

El costo de coordinación (el tiempo-persona consumido de manera concurrente por todo el grupo discutiendo requerimientos individuales) se eleva a niveles inaceptables para la rentabilidad de la organización.

Para mitigar este problema, equipos ágiles avanzados optan por sesiones de estimación reducidas (con un subconjunto representativo de al menos tres estimadores que garantizan la consistencia inicial) o por procesos adaptados como los implementados por Kniberg, donde se delega la estimación detallada a parejas de Senior + Junior y el resto del equipo actúa como revisor técnico cruzado en ciclos breves de retroalimentación.

Tema 4: Progreso y Capacidad de Ejecución: Velocidad y Capacidad

Definición Formal de Velocidad y la Regla del "Todo o Nada"

La **Velocidad** (Velocity) es una métrica fundamental del proceso que mide la tasa de progreso real de un equipo ágil. Se define cuantitativamente como:

La **suma del tamaño** (expresado en Story Points) de todas aquellas Historias de Usuario completadas exitosamente por el equipo durante una iteración determinada.

Para calcular la velocidad real al final de una iteración, se aplica de manera estricta e inflexible la regla del "todo o nada" (all-or-nothing rule):

Solo se contabilizan los Story Points de aquellas historias que han sido completadas en su totalidad, cumpliendo rigurosamente con la Definición de Terminado (Definition of Done - DoD) y habiendo superado con éxito todas sus pruebas de aceptación (Acceptance Tests).

Las historias completadas parcialmente (incluso si se encuentran al 90% de desarrollo) aportan exactamente CERO puntos a la velocidad real de la iteración.

Esta regla busca erradicar la típica disfunción de la gestión tradicional del "código casi listo" o "casi terminado", la cual enmascara retrasos severos de integración y pruebas. El valor de una funcionalidad medio hecha es nulo para el negocio; solo aporta valor real el software que se encuentra en un estado de "potencialmente entregable".

Adicionalmente, no se permite recalcular o modificar los puntos de una historia después de finalizada la iteración. Si una historia se estimó originalmente en 4 SP pero durante el desarrollo resultó ser sumamente más costosa (equivalente a 7 SP), esta aporta exactamente 4 puntos a la velocidad ganada en esa iteración. La variación real quedará absorbida y se balanceaba de forma estadística en las iteraciones futuras.

El Mecanismo de Autocorrección de la Velocidad

La velocidad actúa como un potente filtro corrector de los desvíos naturales de estimación del equipo de desarrollo.

Si un equipo de desarrollo cae en el sesgo sistemático del optimismo y sobreestima su capacidad real al inicio del proyecto (por ejemplo, asumiendo un plan de iteración de 30 SP cuando en la práctica solo tiene capacidad para desarrollar 15 SP), la velocidad real medida al final del Sprint actuará como un baño de realidad empírica:

Al finalizar el Sprint 1, el equipo solo logra completar historias que suman 16 SP debido a cuellos de botella no previstos y fricción técnica.

Al planificar el Sprint 2, la velocidad real del Sprint anterior (16 SP) sirve como el límite de capacidad empírico recomendado para comprometer un nuevo trabajo.

Este mecanismo se conoce en la agilidad como el "**tiempo que hizo ayer**" (yesterday's weather): asumir de forma empírica que el equipo completará en la iteración actual una cantidad de trabajo similar a la completada en las iteraciones de longitud idéntica inmediatamente anteriores.

De esta forma, la velocidad autocorrigió de manera automática las imprecisiones de estimación en el mediano plazo.

Para predecir el comportamiento futuro y armar planes de lanzamiento (Release Plans) realistas, se aconseja estimar la velocidad como un rango probabilístico utilizando tres métricas históricas de base:

- La velocidad de la iteración más reciente.
- La velocidad promedio de las iteraciones completadas del proyecto [68].
- La velocidad promedio de las tres iteraciones más lentas (peor escenario) [68].

Capacidad (Capacity) e Iteraciones

A diferencia de la velocidad (que es una medida empírica del rendimiento pasado expresada en Story Points), la **Capacidad** (Capacity) es una medida analítica de la disponibilidad de recursos de tiempo real de los que dispone el equipo de desarrollo para la iteración que se inicia.

La capacidad se calcula mediante la fórmula estándar de la cátedra:

Capacidad = Horas de Trabajo Disponibles por Día por Desarrollador * Días Hábiles Disponibles en la Iteración

Este cálculo exige detallar de forma rigurosa la disponibilidad de cada miembro del equipo de cara al Sprint, restando días de vacaciones programadas, licencias médicas, feriados calendarizados y el tiempo asignado a reuniones ceremoniales de Scrum o capacitación corporativa.

El Factor de Dedicación (Focus Factor) y la Planificación de Sprints

Para vincular de forma analítica la capacidad de tiempo real (en horas-hombre) con los Story Points a comprometer en una iteración, se introduce el Factor de Dedicación (Focus Factor / Dedication Factor):

Factor de Dedicación = Velocidad Real de las Iteraciones Previas (en SP) / Días-Hombre Totales Disponibles del Equipo en esas Iteraciones

El factor de dedicación representa la proporción del tiempo total de trabajo del equipo que realmente se dedica a la implementación directa de las historias de usuario del backlog.

Nunca equivale al 100%, ya que parte del tiempo diario se consume inevitablemente en actividades de soporte no planificadas, soporte operativo, cambios de contexto, revisión de correos, mantenimiento de equipos o interrupciones informales.

En la industria, el factor de dedicación de un equipo maduro suele oscilar en torno al 70%, aunque ante entornos de alta fricción o infraestructura degradada puede caer de forma drástica al 50% o 40%.

Tema 5: El Enfoque Kanban y la Previsión Probabilística

La Estimación Detallada como Desperdicio (Waste) y el Movimiento #NoEstimates

El marco Lean de desarrollo de software (inspirado en el sistema de producción de Toyota) clasifica como desperdicio (muda / waste) a toda aquella actividad humana que consume recursos y tiempo dentro del proyecto pero que no añade valor real directo al cliente final.

Bajo esta estricta filosofía Lean, pensadores de la comunidad ágil y de Kanban (como David Anderson y Dragos) cuestionan con severidad la necesidad y eficiencia de las sesiones detalladas de estimación y el Planning Poker:

En entornos reales de operación y mantenimiento de software, se descubrió que los equipos de ingenieros dedicaban entre el 33% y el 40% de su capacidad de trabajo neta diaria exclusivamente a analizar y preparar estimaciones precisas de requerimientos futuros para el cliente.

Este tiempo dedicado a estimar reducía de manera drástica la capacidad real del equipo para codificar, probar y entregar código funcional terminado.

Además, las estimaciones detalladas grupales suelen introducir un alto costo de coordinación en la organización.

Este análisis dio origen al movimiento #NoEstimates, el cual propone eliminar por completo la estimación detallada a priori para la mayoría de las clases de servicio de TI. En su lugar, se adoptan dos enfoques Lean alternativos de alta eficiencia operativa:

Descomposición Homogénea: Descomponer todos los requerimientos y solicitudes de trabajo entrantes en porciones de tamaño aproximadamente equivalente (por ejemplo, que cada ticket consuma en promedio entre 0.5 y 2 días hábiles de esfuerzo). De este modo, la velocidad del equipo puede medirse de forma simplificada simplemente contando la cantidad neta de ítems completados por unidad de tiempo, eliminando la burocracia de pesar cada ticket en Story Points (la analogía de Kniberg de "contar ladrillos en lugar de pesar cada ladrillo" cuando su tamaño relativo ya es homogéneo).

Mapeo de Flujo y Previsibilidad de Sistemas: Enfocar los esfuerzos gerenciales en optimizar el flujo de valor a través de políticas explícitas, la limitación del Trabajo en Curso (WIP) y el control de las métricas de flujo históricas.

Métricas de Flujo en Kanban

Para gestionar un sistema de desarrollo sin estimar estimaciones de esfuerzo, Kanban propone medir el flujo real del sistema mediante cuatro métricas cuantitativas clave:

Lead Time (Tiempo de Entrega / Plazo de Ejecución): Es el tiempo total transcurrido desde que un requerimiento o ítem de trabajo es formalmente solicitado e ingresado a la cola por el cliente hasta el momento en que es entregado en producción en su estado final. Se mide de forma determinista en días de trabajo reales. Es la métrica por defecto para evaluar la eficiencia y la satisfacción del cliente.

Touch Time (Tiempo de Tocado / Tiempo de Trabajo Neto): Es el tiempo real en días de trabajo en el cual el equipo de desarrollo estuvo operando físicamente sobre el ítem de trabajo de manera activa. Excluye todos los periodos de espera en colas, estados de bloqueo por dependencias externas, o buffers de inactividad.

Eficiencia del Ciclo de Proceso: Es la relación porcentual entre el tiempo de trabajo neto y el plazo de ejecución total del ítem: $\text{Eficiencia del Ciclo} = \text{Touch Time} / \text{Lead Time} * 100\%$

Nota: En la mayoría de las organizaciones tradicionales de TI, la eficiencia del ciclo del proceso es sorprendentemente baja (inferior al 10%), demostrando que el retraso en los proyectos proviene mayormente de la inactividad de los tickets en colas y buffers de espera, y no de la velocidad de codificación de los programadores.

Throughput (Tasa de Rendimiento): Es el número bruto de ítems de trabajo completados y entregados de forma exitosa por unidad de tiempo dada (ejemplo: tickets completados por semana). Sirve como el indicador objetivo de la productividad operativa y de la mejora continua del sistema.

Previsión Probabilística mediante Simulaciones de Monte Carlo

Al eliminar la estimación determinista tradicional basada en Story Points y horas, ¿cómo puede un ingeniero en sistemas dar previsibilidad y fechas de lanzamiento creíbles al cliente?

La respuesta técnica de Kanban radica en la Previsión Probabilística empleando Simulaciones de Monte Carlo:

En lugar de sumar estimaciones individuales subjetivas propensas a errores, el sistema utiliza los datos históricos de flujo reales acumulados por la organización (tasas reales de Throughput y variabilidad de Lead Time).

Un algoritmo informático ejecuta un modelo de simulación matemática miles de veces de forma automatizada, simulando escenarios alternativos y combinaciones dinámicas de tamaño de ítems, tiempos de entrega y tasas de arribo de fallas de la vida real.

El resultado de la simulación de Monte Carlo no es una fecha fija (promesa determinista que suele violarse), sino una distribución de probabilidad de fechas de finalización y rangos de entrega.

Esto permite al equipo gestionar las expectativas del cliente y los acuerdos de nivel de servicio (SLA) bajo un esquema probabilístico de control de riesgos sumamente confiable.